Assignment # 2 – Experimental Analysis of RNNs and LSTMs for Next Word Prediction

Instructor: Dr. Ahmad Raza Shahid

Course Information

- **Course**: Deep Learning for Perception (CS4045)
- **Assignment**: Analysis of RNNs and LSTMs for Next Word Prediction
- **Due Date**: As announced on Google Classroom (GCR)
- **Total Points**: 150
- **Format**: Jupyter / Colab / Kaggle Notebooks + Formal Report (PDF)

Assignment Philosophy

This assignment is experimental and research oriented. Instead of following a fixed architecture, you must systematically experiment with Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks. You are expected to determine the optimal depth, hidden units, and hyperparameters yourself to solve complex sequential problems.

Dataset Overview

Primary corpus (training): Jane Austen novels and Tiny Shakespeare

- **Jane Austen text**: Kaggle dataset
  `https://www.kaggle.com/datasets/joshmcadams/jane-austin-and-charles-dickens`
- **Tiny Shakespeare** (approximately 40,000 lines of play text)
  `https://www.kaggle.com/datasets/thedevastator/the-bards-best-a-character-modeling-dataset`

Secondary corpus (out of domain)

- **BBC News full text articles**: 2,225 articles in five categories (2004–2005)
  `https://www.kaggle.com/datasets/shivamkushwaha/bbc-full-text-document-classification`

Phases & Tasks

- The Austen file still contains some Gutenberg noise. Cleaning it is part of Phase 1.
- The BBC download contains two copies of every article (about 4,450 files for 2,225 articles). Remove the duplicates before you use it.
- Split the primary corpus (Austen and Shakespeare) into 80% train / 10% validation / 10% test before building the vocabulary or training any model. Split by whole sentences (or blocks of consecutive sentences), not by individual windows. Use a fixed random seed.

- Window size: use $k = 20$ for Phases 3–6 (Phase 2 varies k).

Phase 1 – Sequential Data Engineering (15 points)

Goal: Transform raw text into a format suitable for recurrent layers.

- Implement a sliding window tokenizer. Convert text into sequences of k words as input with the next word as target (for example, "it is a truth" with $k = 3$ gives input "it is a" and target "truth").
- Build the vocabulary from the training split only. Words below a minimum count (state it) become `<UNK>`. Report the vocabulary size and the `<UNK>` rate on the train, validation and test splits.
- Add `<s>` and `</s>` tokens at sentence boundaries.

Phase 2 – The Vanishing Gradient Experiment (25 points)

Research question: How do Simple RNNs and LSTMs differ in keeping long term dependencies?

- Train a Simple RNN and an LSTM on the same next word prediction task, with the same embedding size, hidden size, optimizer and number of epochs (state the values you chose).
- Compare their performance as the window size increases: repeat for $k = 5, 10, 20, 40$ and 80 and plot validation perplexity against k for both models. Evaluate all values of k on the same target words, so the perplexities are comparable.

Phase 3 – Hyperparameter & Architecture Research (10 points)

Goal: Find the best architecture through systematic testing.

- You decide the number of layers (1 to 3), hidden units (128, 256, 512) and dropout (0, 0.2, 0.5).
- Run at least 9 experiments with early stopping on validation perplexity. At least 2 of them must be Simple RNN configurations.
- **Model selection:** the best RNN and the best LSTM are the ones with the lowest *validation* perplexity. Report test perplexity for every run, but do not use it to choose a model. These two models are used in Phases 4, 5 and 6.

Phase 4 – Next Word Prediction Chatbot (30 points)

Goal: Use your best LSTM model to generate text interactively.

- Build a chatbot from your best LSTM.
- Write an `input()` loop where the user enters a seed phrase and a number N, and the model generates the next N words. Typing `quit` should end the loop. Invalid input must not crash the loop.
- Support greedy decoding (always the most probable word) and temperature sampling, and let the user choose.

Phase 5 – Perplexity and Confidence Analysis (20 points)

Research question: Are some sentences harder for the model than others, and can we tell from simple features?

- Take the probability your model gives to the correct next word at each position and compute the perplexity of every test sentence (a sentence runs from `<s>` to `</s>`). Do this for your best RNN and best LSTM.
- Split the test sentences into three equal-sized groups (short, medium, long) by length and compare RNN and LSTM perplexity in each group.
- Fit a linear regression that predicts sentence log-perplexity from sentence length and share of rare words (a word is rare if it occurs fewer than 50 times in the training split; `<UNK>` counts as rare). Fit it on 70% of the test sentences and report R^2 and RMSE on the remaining 30%.

Perplexity of a sentence with N predicted tokens (count `</s>` as a predicted token; `<s>` is never predicted):

$$PP(W) = P(w_1, w_2, \ldots, w_N)^{-1/N}$$

Phase 6 – Multi Style NLP Adaptation (40 points)

Goal: Investigate model adaptability across different linguistic distributions.

- **Task:** Take your model trained on the primary corpus (Austen and Tiny Shakespeare) and test its next word prediction on a completely different style: the BBC News articles provided above.
- **Requirement:** Calculate the change in perplexity when the model encounters this out of domain text.

Report Expectations: Discuss the "Generalization Gap" and how specialized vs. generalized training data influences model performance.

Report (10 points)

Submit a formal report including:

- Abstract and Introduction
- Methods: summary of your experiments and architecture choices
- Metrics: perplexity and accuracy, with plots
- Discussion: interpret the findings and the impact of the confidence analysis
- Conclusion: what you learned

Instructions

- Submit a single ZIP file named: `i22xxxx_name_A2.zip`
- The ZIP file must include:
  - `notebooks/` folder (all experiments, reproducible, fixed random seed)

  - `report.pdf`
  - `README.txt` with instructions to run the code
- Plagiarism = Zero marks
- Late submission = Zero marks
- All code must run without errors (non-runnable code = Zero marks)
- The report is mandatory (no report = no marks)
- Use Google Colab, Kaggle Notebooks or Jupyter Notebook
- Clearly write Name and Roll No at the top of each file

Total = 150 marks